Pramaan

COMPLETE WALKTHROUGH · NOVEMBER 2026

Every sale. A traceable story.

A guide to PRAMAAN, the revenue-audit prototype where the AI investigates, the code computes, the human decides, and everything is provable.

For everyone

Gold “In plain words” boxes explain each part without jargon.

For engineers

Blue “Under the hood” boxes give the implementation detail.

Synthetic data only · not an audit opinion · open source under AGPL-3.0-only

Contents

1 What PRAMAAN is

2 Key ideas, explained simply

3 Who uses it

4 Install and start

5 First-time help built into the app

6 Walkthrough: the client side

6b Bringing in real-world files: Tally, GST returns and PDFs

7 Walkthrough: the auditor side

7b Samples, findings and the evidence behind each

8 The 16 automated checks

9 The AI investigation agent

9b The agent at work, step by step

10 Human review and sign-off

11 Workpapers and export

12 Integrity checks: how tampering is detected

13 Architecture

14 Security and privacy

15 AI assistant and MCP tools

16 Measured results

17 Quality and testing

18 Configuration and command cheat-sheet

19 Limits, roadmap and troubleshooting

How to read this guide. New to auditing or software? Read the gold boxes and look at the screenshots; that is the whole story. Building on PRAMAAN or reviewing it technically? The blue boxes and sections 13–18 cover architecture, security, tests and commands.

1 What PRAMAAN is

PRAMAAN (from the Hindi and Sanskrit word for “proof”) is a prototype tool that helps an auditor check whether a company's reported sales are real, complete and recorded in the right year, and keeps a tamper-evident record of every step.

IN PLAIN WORDS

When a company says “we sold ₹10 crore this year”, an auditor has to check it. Today that means spreadsheets, emailed PDFs and a lot of manual matching. Files can be changed after the fact, and it is hard to prove later what was checked.

PRAMAAN does three things: it **keeps** the company's original records exactly as received, it **checks** them automatically (with an AI helper for investigating), and it leaves the **final judgement to a human auditor**, recording every step so it can be proven later.

What it does

Job	How PRAMAAN does it
Preserve	Every uploaded file is fingerprinted (SHA-256) and stored unchanged. A new version is added alongside the old one, never overwriting it.
Check	16 automated tests (T00–T15) compare the sales register, invoices, dispatch records, journals and bank statements, and pick a reproducible sample to test in detail.
Investigate	An AI agent follows up one exception at a time. Code does all the arithmetic, the server decides the outcome, and the agent can only propose actions.
Decide	Only a person holding a separate human-review key can record a final decision. AI and auditor keys can only propose.
Prove	Every action goes into a hash-chained log. Exports are sealed packages that show if anything inside was changed.

What it is not

- It is **not an audit opinion** and never says “fraud”. It produces review leads, and a qualified person concludes.
- It is a **prototype**: one company, one computer, synthetic (made-up) data. It is not yet a hosted product.
- Its e-invoice signature check uses a **labelled demo key**, not an official government (GST/IRP) connection.

UNDER THE HOOD

Stack: **FastAPI** (Python) backend, **SQLite** in WAL mode, a **content-addressed vault** (files stored under their SHA-256), a **hash-chained, append-only event log** enforced by database triggers, and a per-role **SQLite authorizer**. Frontend: **React 19 + Vite + TypeScript**. AI: an OpenAI-compatible client (Gemini by default) that reaches data only through an **MCP** tool server, which calls the same authenticated API as people do. Licence: AGPL-3.0-only.

2 Key ideas, explained simply

Seven ideas appear everywhere in PRAMAAN. If you understand these, you understand the product.

Idea	Plain meaning	Everyday comparison
Fingerprint (SHA-256)	A code computed from a file's exact contents. Change one character and the code changes completely.	A seal on an envelope: if it is broken, you know someone opened it.
Snapshot	A frozen copy of all the data taken when an audit run starts. Later edits to the books don't change it.	A photo of a room: rearranging the furniture later doesn't change the photo.
Hash chain	Each log entry contains the fingerprint of the entry before it, so deleting or editing history breaks the chain.	Numbered pages glued together: tear one out and the gap shows.
Observation / finding	An automated signal that something may need a look, tied to a record and the rule that raised it. Never a conclusion.	A smoke alarm: it tells you to check, not that there is a fire.
Sample	The invoices picked for detailed testing: high-risk ones always, the rest at random with a recorded seed so the pick can be repeated.	Spot-checking bags at an airport, with the selection method written down.
Proposal	A suggested decision or action (from the auditor or the AI) that does nothing until a human confirms it.	A draft email that only goes out when you press Send.
Human review	The final decision on a finding, recordable only with the separate human-review key.	A manager's signature on an expense claim.

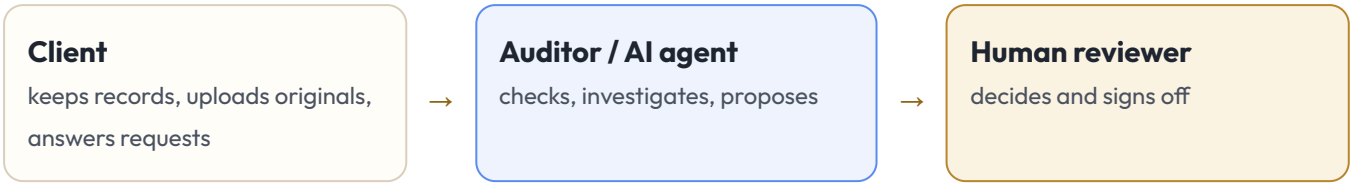
IN THE APP

Each of these words has a small **?** next to it on screen. Hover over it or tap it for the explanation, and **Help & guide** in the account menu lists all of them.

3 Who uses it

PRAMAAN has three roles. Each person signs in with a **key** (a long code) that decides what they can see and do. The server checks the key on every single request, so hiding a button in the screen is never the only protection.

Role	Who they are	Can do	Cannot do
Client	The company being audited	Record sales, upload the original sales register, related-party list and bank statement, answer document requests	See audit findings, run audits, change history
Auditor	The audit team, or an AI agent with an agent key	Reconcile the register, run audits, view findings and samples, investigate, propose decisions, export workpapers	Change the client's books, record a final decision
Human reviewer	The responsible auditor (a person)	Everything an auditor can, plus record final decisions and confirm the agent's proposals	Use this key from an AI tool: it is refused on the AI channel



UNDER THE HOOD

Each FastAPI route has a `name` that maps to an action in `backend/security.py`. A middleware resolves the route (in a way that works on both old and new FastAPI/Starlette routing), authenticates the bearer key (stored only as a hash, with expiry and revocation) and checks the action against the role's allow-list. Unknown routes are denied. `record_review_decision`, `confirm_agent_action` and `reject_agent_action` are human-only and blocked on the MCP channel. Agent keys (`--agent`) carry the agent policy inside the key itself, whatever headers a client sends.

4 Install and start

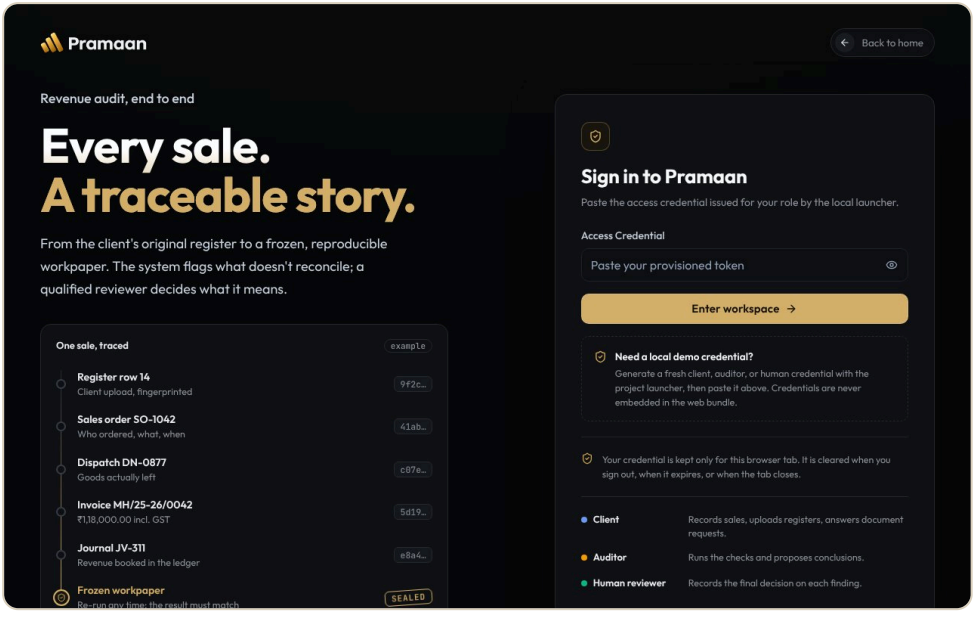
> IN PLAIN WORDS
>
> You need two free programs, Python and Node.js. After that you double-click one file and the app opens in your browser. Everything runs on your own computer.

One-time setup

1. Install **Python 3.11+** (python.org; on Windows tick “Add Python to PATH”).
2. Install **Node.js 20.19+ or 22.12+** (nodejs.org).

Every time

1. **Windows:** double-click `START_PRAMAAN.bat`. **macOS/Linux:** run `./run-local.sh`.
2. The launcher installs anything missing (the first run takes a few minutes), seeds the demo data, starts the API (port 8000) and the website (port 5173), opens the browser, and opens a Notepad file with three sign-in keys valid for 8 hours.
3. Paste the key for your role into **Access credential** and press **Enter workspace**.

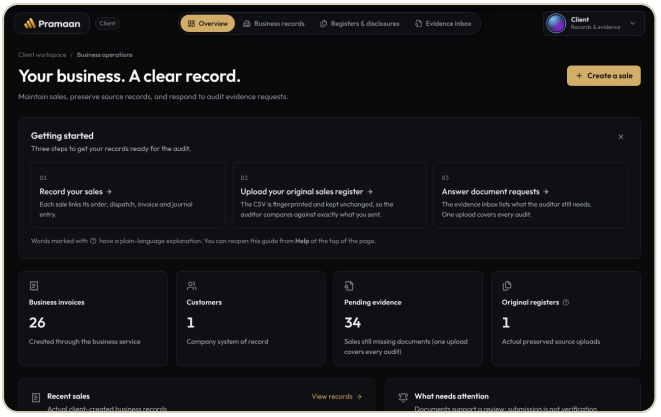


The sign-in page. Keys are kept only for the browser tab and are never built into the website.

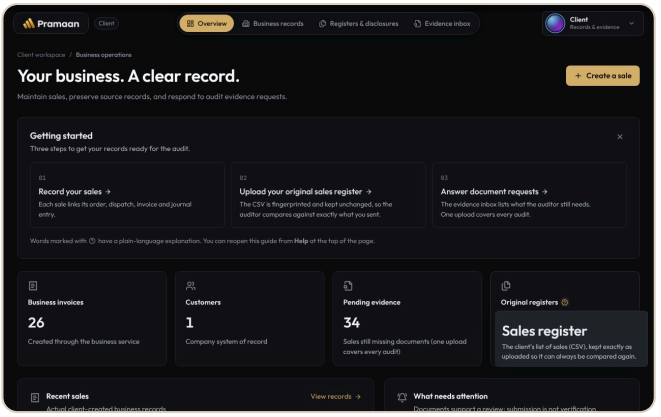
> UNDER THE HOOD
>
> The code imports itself as `Pramaan.backend`. The tracked `launcher/` shim maps that name to the repository root whatever the folder is called, so commands run outside the launcher need `PYTHONPATH` set to that folder first (`$env:PYTHONPATH = "$PWD\launcher"`). Demo data lives in `.demo/runtime` (git-ignored), and the launcher never overwrites it. Docker: `docker compose up --build`, then open http://127.0.0.1:8080.

5 First-time help built into the app

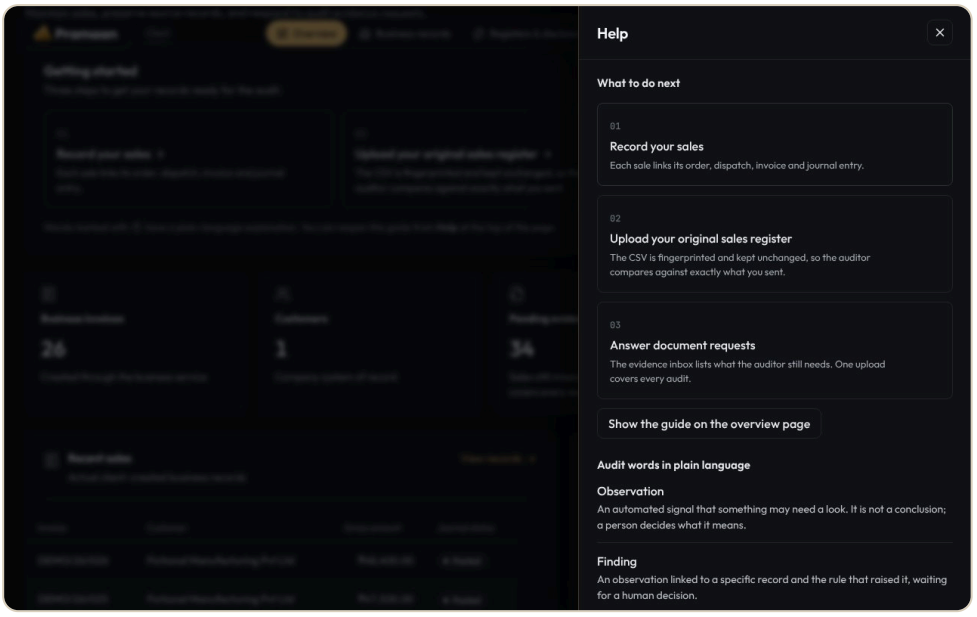
Audit software is full of jargon, so PRAMAAN explains itself in three ways.



Getting started card: the steps for your role, each linked. Hide it with ✕ and bring it back from Help.



A “?” hint: hover over it, tap it or press Enter for a one-line meaning. It stays inside the screen.



Help (open your account menu, top right, then Help & guide): what to do next, plus every audit word in plain language.

Empty tables also say what to do next, for example *“No register uploaded yet. Upload your original sales register CSV above.”*, instead of a blank “No records”.

UNDER THE HOOD

All definitions come from one glossary module (`web/src/glossary.tsx`), so no word has two explanations. The hint is a button with an accessible name (“What does … mean?”) and no visible text, so labels and test selectors are unchanged. It toggles a `role="note"` popup, closes on Escape or an outside click, and shifts itself to stay in the viewport. The guide's dismissal is stored per role in `localStorage` (wrapped in try/catch). Everything passes axe (WCAG 2.1 AA) with 0 violations.

6 Walkthrough: the client side

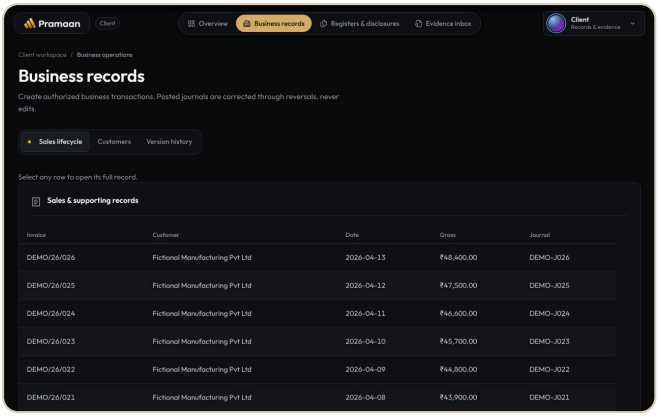
IN PLAIN WORDS

The client's job is simple: keep proper records, hand over the original files, and send documents when the auditor asks. Nothing the client uploads can be quietly changed later; a new file becomes a new version.

6.1 Overview

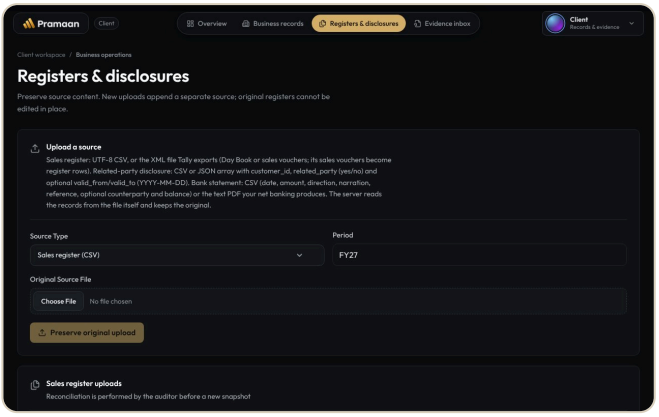
Counts of invoices, customers, outstanding documents and preserved registers; recent sales; what needs attention; and an activity trail of everything recorded, in plain words (“Sales register uploaded”).

6.2 Business records



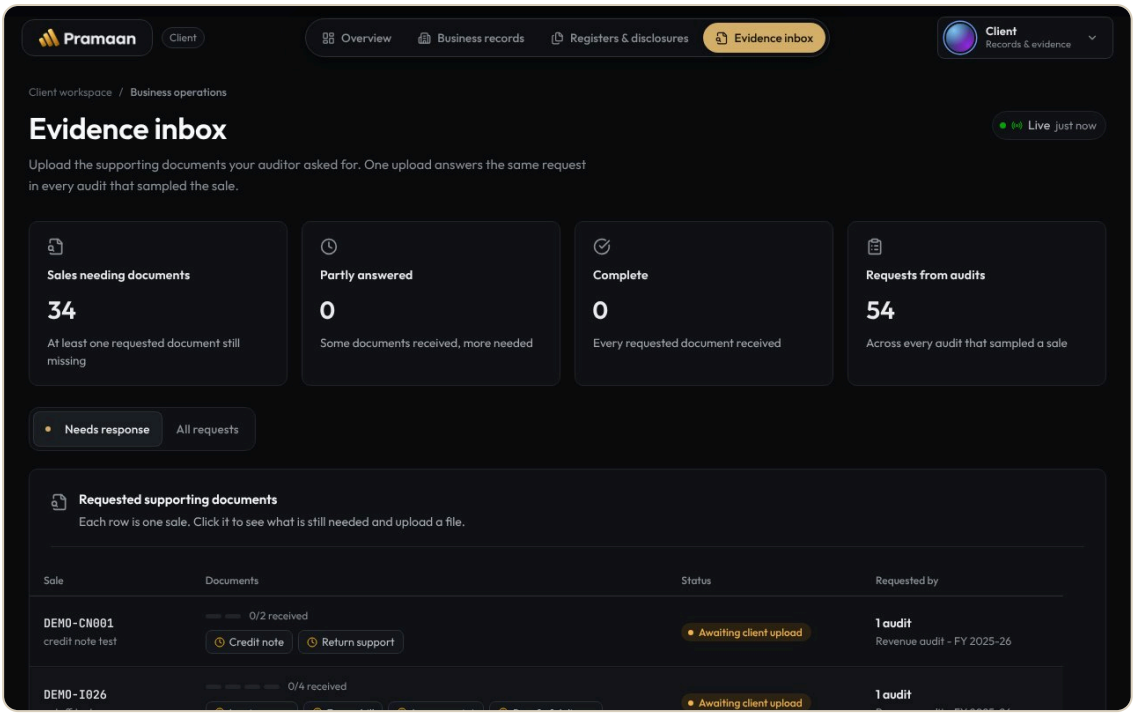
Business records: each sale links its order, dispatch, invoice and journal entry. Posted journals are never edited; a correction adds a reversing entry with a reason.

6.3 Registers & disclosures



Upload a source: the sales register (CSV or a Tally XML export), related-party disclosures (CSV/JSON), or a bank statement (CSV or text PDF) for the AI agent to trace payments.

6.4 Evidence inbox



Evidence inbox: each row is one sale the auditor needs documents for (invoice copy, e-way bill, lorry receipt, proof of delivery…). One upload answers that request for every audit. Invoice PDFs are read on arrival (number, date, total).

UNDER THE HOOD

Uploads are base64 JSON, capped at 10 MiB decoded (larger bodies get HTTP 413 before parsing). Bytes go to the vault under their SHA-256, parsed rows go to SQLite, and a `register_imported` event is appended to the hash chain. Related parties are parsed from the uploaded file, not taken from client-supplied JSON. Business tables use append-only triggers, and a correction is a reversing journal with a 10–500 character reason, recorded with the actor.

6b Bringing in real-world files: Tally, GST returns and PDFs

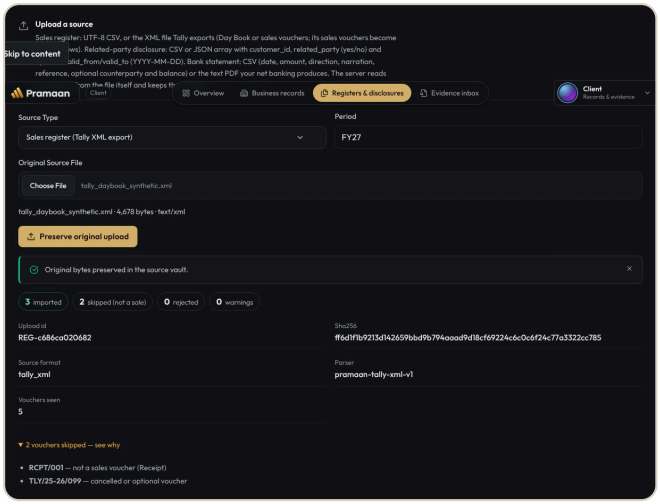
IN PLAIN WORDS

Most companies keep their books in **Tally** and get bank statements and invoices as **PDFs**. PRAMAAN reads all three and then **checks** them. It never guesses and never fixes anything quietly: whatever it can't read, or whatever doesn't add up, is listed with the reason. The original file is always kept unchanged.

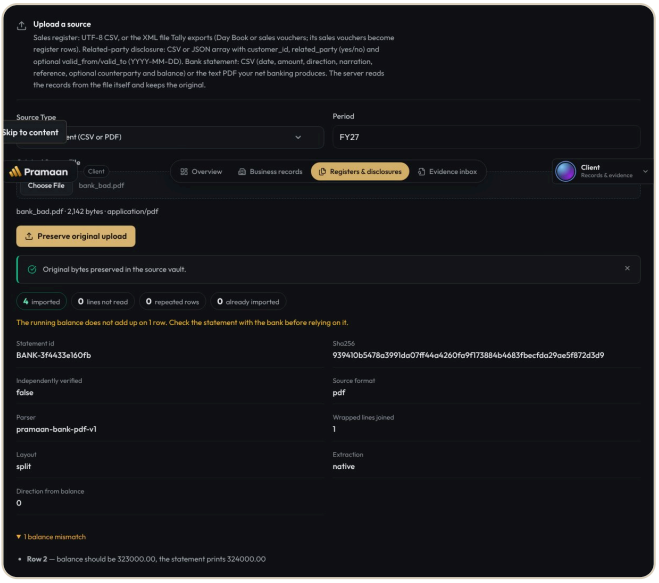
File	What PRAMAAN reads	What PRAMAAN checks
Tally XML export TallyPrime: Export → Day Book, XML	Each sales voucher becomes a register row (bill no., date, customer, amount, GSTIN). The XML file is the preserved register.	Counts **imported**, **skipped** (not a sale), **rejected** (a sale it can't read safely) and **warnings**: wrong GSTIN check digit, a repeated voucher number, entries that don't balance.
GSTR-1 return JSON downloaded from the GST portal	Each invoice, export and credit/debit note becomes a register row (number, date, buyer GSTIN, value, taxes). B2CS totals are summarised, not matched. No portal login.	Value = taxable + tax? Dated inside the return period? GSTIN check digit, repeated numbers. Then reconciled against the books like any register: **what was reported to GST vs what is in the books**.
Bank statement PDF from net banking, or CSV	Two layouts: separate Debit / Credit columns, or **one Amount column with Dr/Cr** (or a minus sign, or only the balance). Any column order, several pages, wrapped narration, value dates, overdrawn (Dr) balances. Every row remembers its page and line.	Does the **running balance** add up on every row? Does this statement **start where the last one ended**? The **same file twice** is refused, and repeated transactions are counted once.
Invoice PDF Uploaded as the "Invoice copy"	Invoice no., date, GSTINs, taxable value, CGST/SGST/IGST, round-off and total, each with its page number.	Does taxable value + tax equal the total? Does it match the **sales register**? Two different values printed for one field are marked **ambiguous**, not picked.
Scanned PDF Needs Tesseract installed	Read by OCR on the same computer; nothing is sent anywhere.	Words OCR is unsure of are **never imported**. Every OCR value is labelled **OCR — check** until a human reviewer confirms it.
Wrong file uploaded	*Replace it* with a corrected file and a reason.	The corrected file is **version 2**; the original and its fingerprint are kept. Audits already frozen are unchanged and still reproduce exactly.

IN PLAIN WORDS

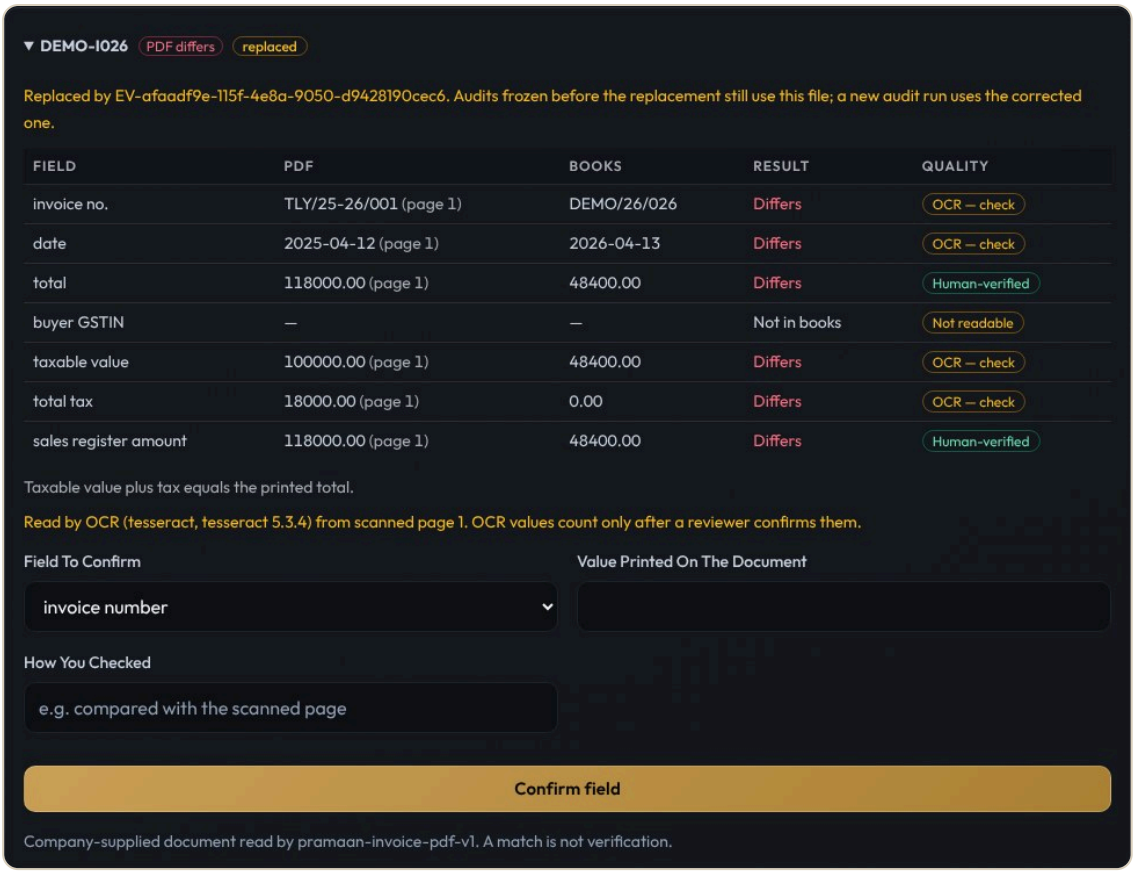
How sure is each value? Instead of a made-up percentage, every field gets one of six labels decided by a fixed rule: **Exact** (read from the PDF text), **Validated** (exact, and an independent check agrees, such as the GST maths or the GSTIN check digit), **OCR — check**, **Ambiguous** (two different values printed), **Not readable**, or **Human-verified** (a reviewer looked at the file and recorded the value, with their name and a note).



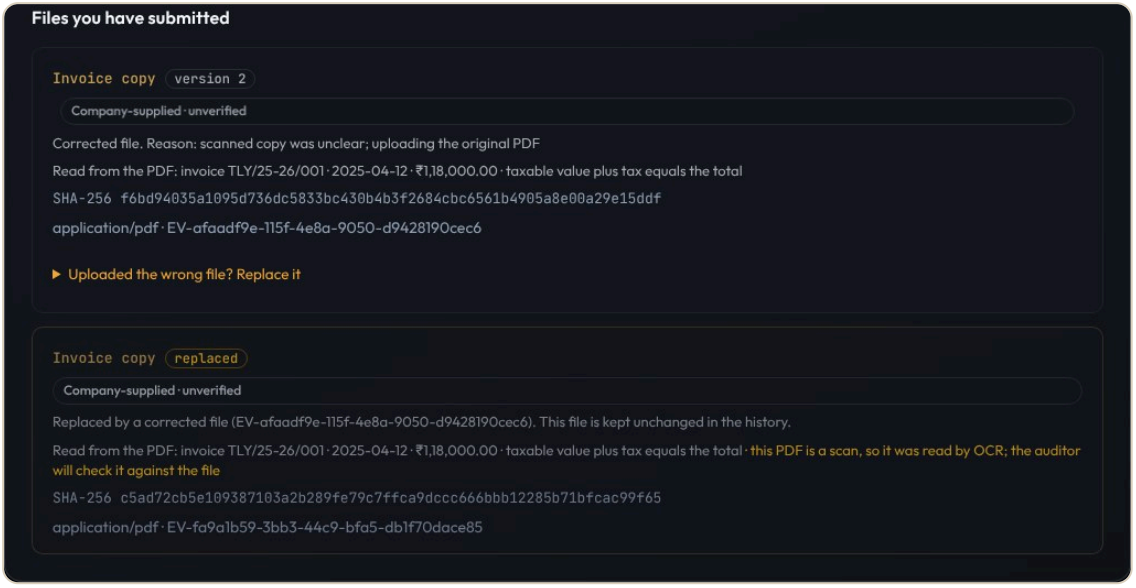
Tally import: counts at the top. A receipt and a cancelled voucher were skipped, and the reason is shown.



Bank PDF with one wrong printed balance: the import goes through and the mismatch is listed for the auditor.



A scanned invoice read by OCR: each field shows its quality label. The total has been confirmed by a human reviewer; the rest still say “OCR — check”. This file was later replaced by a corrected one.



Client view after a replacement: the original file is marked “replaced” and kept; the corrected file is version 2, with the reason.

UNDER THE HOOD

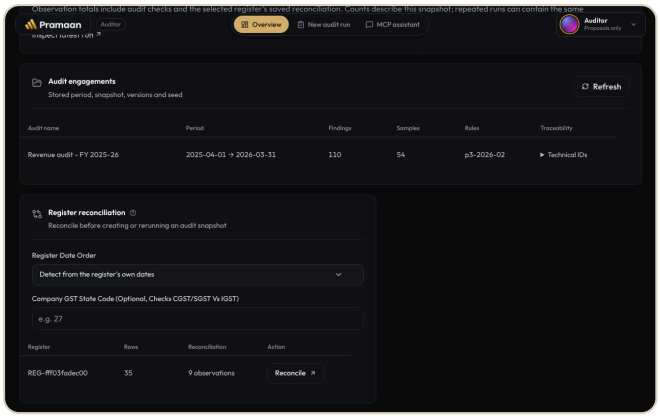
`backend/document_import.py` (no AI anywhere). Tally: ElementTree; a DOCTYPE/ENTITY anywhere in the file is refused; each import report is kept in full in `import_reports`, with its hash in the event log. Bank PDF: PyMuPDF word coordinates; `check_balances` works oldest- or newest-first; a repeat SHA-256 gets HTTP 409, and the agent de-duplicates across statements by date, amount, direction and reference. Invoice: visual-line text, labelled patterns, page refs, GST arithmetic (exact with a printed round-off, otherwise ±50 paise). Readings are append-only in `p3_document_reads`, and a reader crash never loses the upload. Round 2 added: a single-Amount layout with direction from Dr/Cr, sign or proven balance change; `backend/ocr.py` (local Tesseract, 20 pages, 30 s/page, 120 s/document, words under confidence 80 held back); append-only `p3_evidence_versions`, `p3_field_verifications`, `p6_bank_row_sources` and `import_reports` (the event log stores only payload hashes); statement-to-statement continuity; and a Tally HTTP/XML fetch command, tested against a stand-in server only. GSTR-1 JSON: `backend/gstr1.py`, tested on synthetic returns only. Not covered: a verified live Tally link, a live GST portal link, SAP, and real (non-synthetic) documents. Full tracker: `docs/IMPORT_AUDIT.md`.

7 Walkthrough: the auditor side

IN PLAIN WORDS

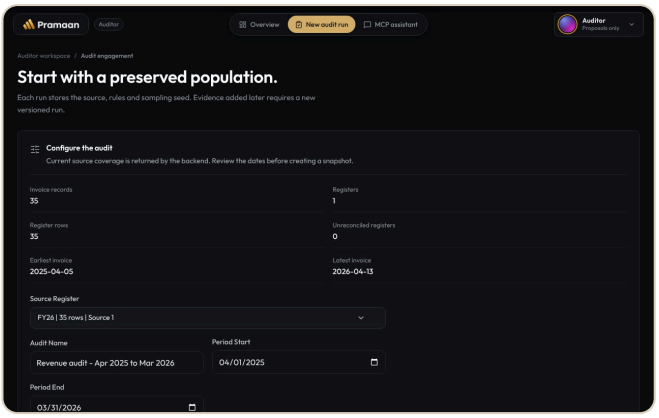
The auditor first checks that the client's list of sales matches the books, then starts an **audit run**, a frozen photo of the data that every check works on. The results are sorted so the most serious come first.

7.1 Reconcile the register



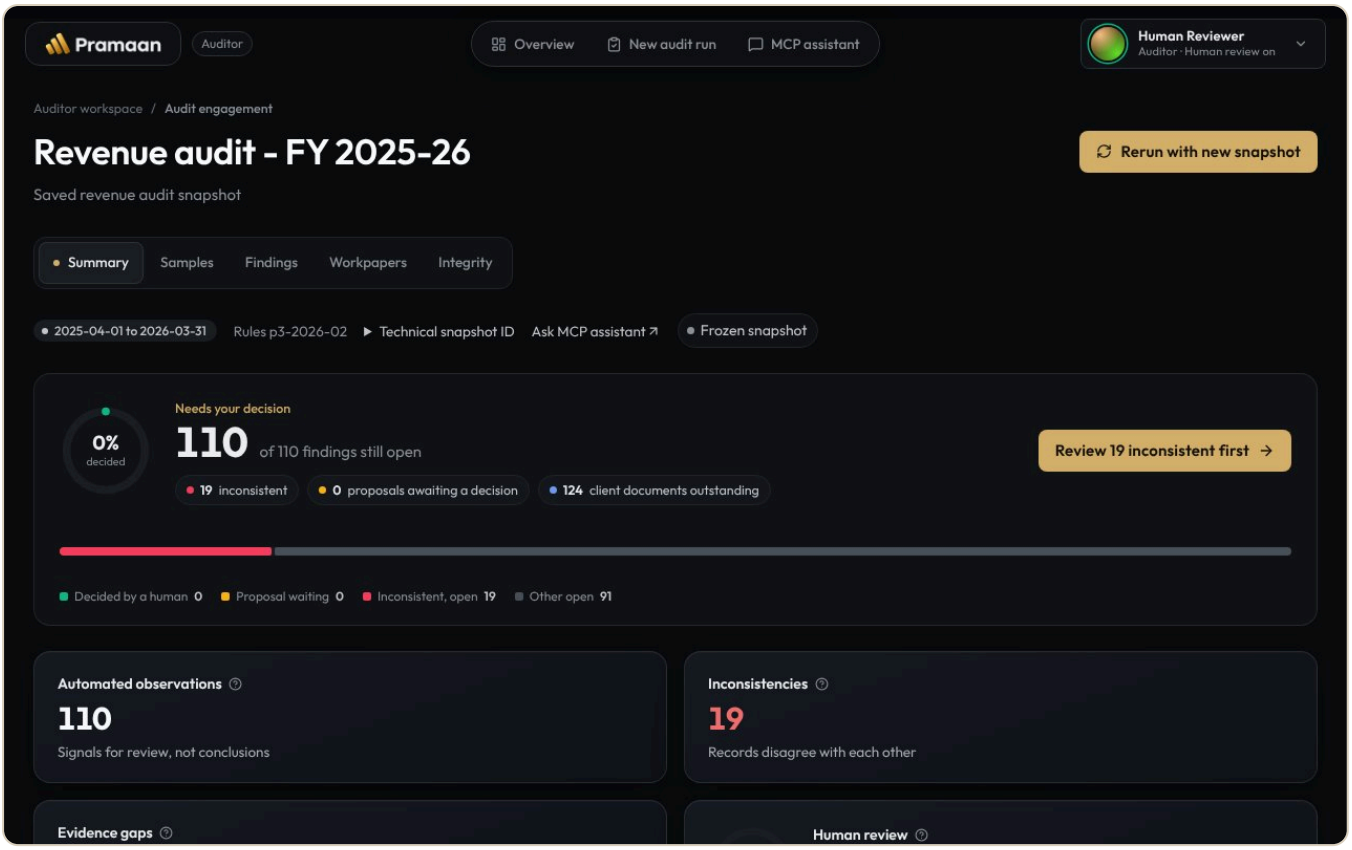
Register reconciliation (on the auditor overview): compare the uploaded register line by line with the recorded sales before starting a run.

7.2 Start an audit run



New audit run: choose the reconciled register and the period. The run stores the source, rules, sampler version and sampling seed.

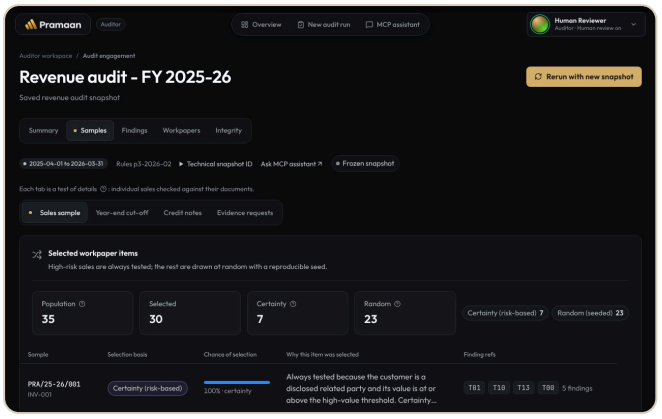
7.3 Run summary



Run summary: “Needs your decision” leads, then signal types, which tests raised them, which assertions are affected, and the amounts flagged (unconfirmed until a human reviews them).

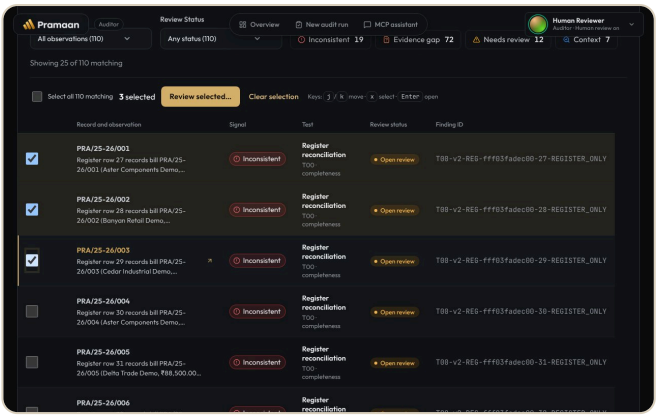
7b Samples, findings and the evidence behind each

7.4 Samples



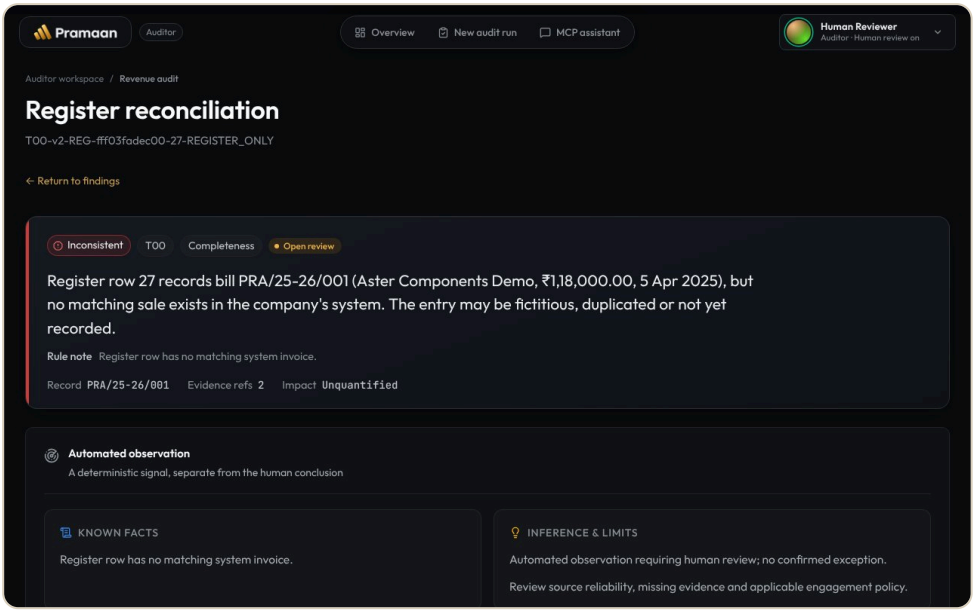
Samples: Sales sample, Year-end cut-off and Credit notes. Each item shows why it was selected (certainty or random), its chance of selection, and the findings linked to it.

7.5 Findings and bulk review



Findings: filter by signal type or review status, select several rows (keyboard: j/k to move, x to select, Enter to open) and review them together.

7.6 A single finding



Finding detail: the observation in plain language, known facts kept separate from inference and limits, the rule, the snapshot it came from, and the agent investigation panel.

UNDER THE HOOD

Sampling uses a persisted hash-ranked seed (by default drawn by the server: `srv-` plus 128 random bits). Certainty items are always included and the rest are probability-selected, with selection reasons, population coverage, ruleset and sampler versions stored. No statistical population projection is claimed. Findings carry a deterministic ID (`F3-` plus a content hash), state, assertion, rule hash and evidence references. Labels in the UI distinguish *automated*, *AI*, *proposal* and *human* statements.

8 The 16 automated checks

IN PLAIN WORDS

Each check is a rule written in code, not AI. It compares records and raises a signal when they disagree or when expected evidence is missing. A signal means “look here”, never “this is fraud”.

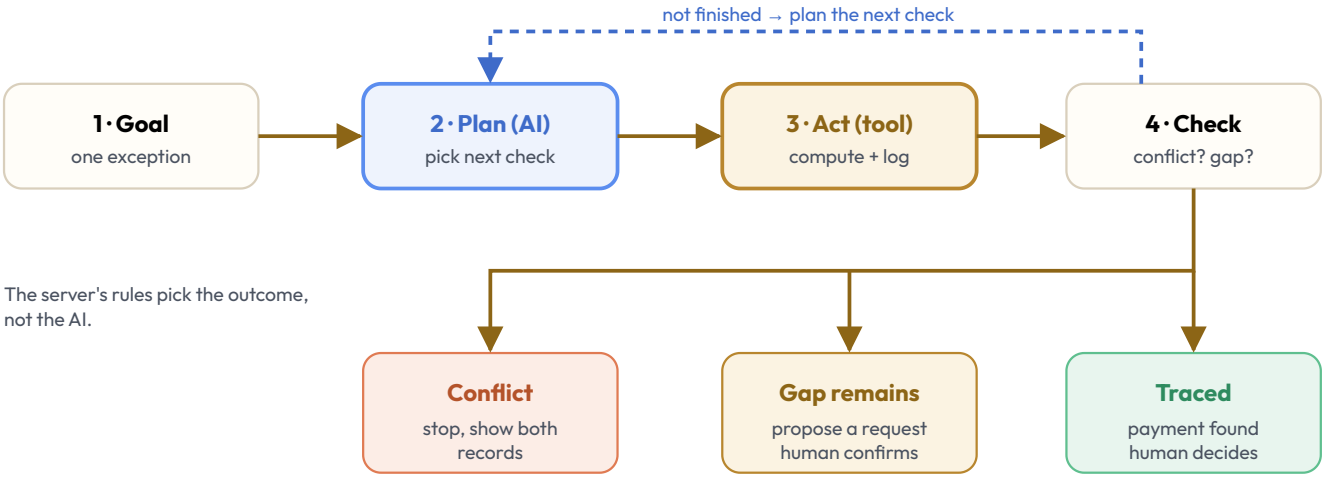
Test	What it checks	How a result is framed
T00	Register vs recorded sales: field-by-field date and exact-amount comparison	A difference or a review gap
T01	Sales to disclosed related parties (dated disclosures)	Classification, not wrongdoing
T02	Related-party network: candidate links and circular trading patterns	A pattern needing corroboration
T03	Revenue cut-off timing around year-end	Review required (needs contract/policy facts)
T04	Missing movement documents (e-way bill, lorry receipt, proof of delivery)	A gap, which does not prove a sale is false
T05	Revenue journals with no invoice reference	A book-record inconsistency
T06	Invoice vs dispatch: value, quantity, fields	An inconsistency when inputs disagree
T07	Reused evidence across different sales	A review pattern
T08	Period-end concentration of sales	A review pattern
T09	Post-period credit notes reversing year-end sales	A review pattern
T10	Transaction splitting below thresholds	A review pattern; no intent inferred
T11	Movement plausibility: route time, vehicle overlap, load, event order, PIN/consignee	“Impossible” only for a supported hard constraint
T12	Requested evidence status per assertion	Company evidence is never treated as independent
T13	E-invoice QR signature (Ed25519) and field match	Demo key only, not official IRP verification
T14	Unapproved promotions and pricing	Review candidates
T15	GSTR-1 return vs books (synthetic)	An inconsistency for review

Signals fold into four types: **Inconsistent** (records disagree), **Evidence gap** (a document is missing or can't be checked), **Needs review** (a pattern worth a look) and **Context** (background). Every one still needs a human decision.

9 The AI investigation agent

IN PLAIN WORDS

For one flagged sale, the AI acts like a junior assistant: it decides what to look at next (the invoice, the register, the bank statement), and the software fetches and calculates the facts. If the records contradict each other, it stops and shows both. If something is still unexplained, it drafts a request to the client, and nothing is sent until a human approves it.

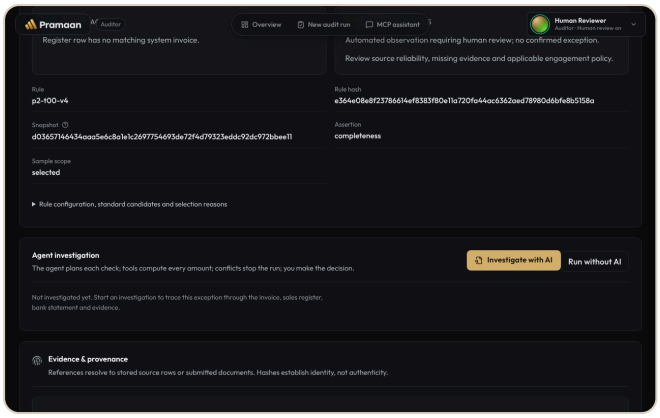


Guardrails (enforced in code, not by asking the AI nicely)

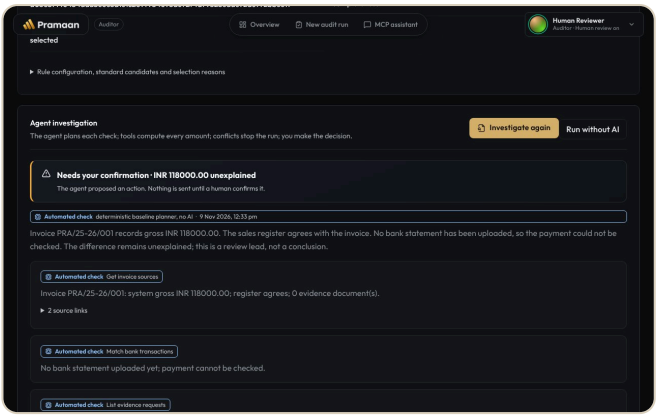
Rule	How it is enforced
Amounts only from tools	Every amount in the AI's text must match a number a tool returned; otherwise the text is rejected and a system summary is shown. The word “fraud” is also rejected.
Names never leave the server	The AI sees record IDs, amounts, dates and masked party references (`PARTY—<hash>`), never names, GSTINs or bank narrations.
Conflicts stop the run	Equal bank matches, an uncertain payer, or register vs invoice disagreement with no support: the server stops and shows both records.
Humans confirm every action	The agent can only *propose*. The proposal is hash-sealed, shown to a human exactly as it will be sent, and can be confirmed once.
It stays on topic	It is scoped to the one invoice under investigation, with a step budget.
Provider problems don't break it	Rate limits, server errors and dropped connections are retried, then it moves to the next fallback model.

9b The agent at work, step by step

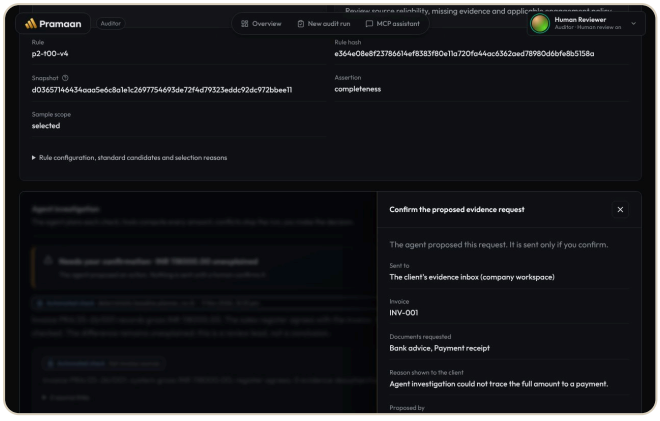
A real run on a planted case. The sales register shows a different amount from the invoice, and the bank backs the invoice.



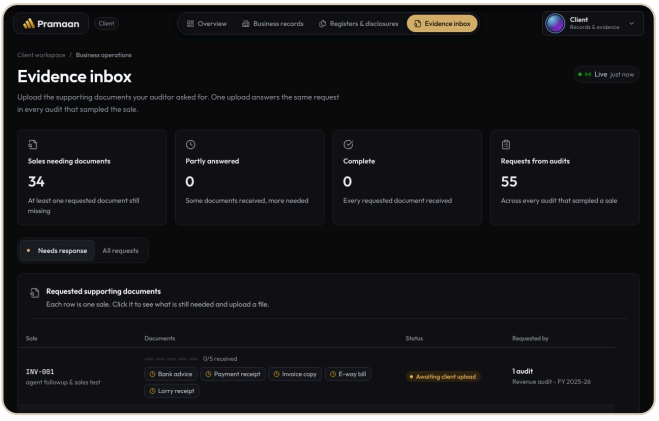
1. On the finding, the auditor starts the investigation (with AI, or “Run without AI”).



2. Each step is labelled with the tool used and its source links. Result: ₹1,18,000 unexplained.



3. Review and confirm: exactly what will be sent, to whom and why. Nothing is sent before this.



4. After the human confirms, the request appears in the client's evidence inbox.

UNDER THE HOOD

Loop: `backend/p6_agent.py`. Deterministic tools (`get_invoice_sources`, `match_bank_transactions`, `match_party`, `list_evidence_requests`, `propose_evidence_request`) compute everything and write each call to the audit log. A deterministic conflict detector and outcome rule set the status: `explained`, `unexplained`, `needs_confirmation`, `conflict` or `budget_exhausted`. The prompt (`investigate-v2`) adds a check-before-finish nudge. The LLM runs through an OpenAI-compatible endpoint (`LLM_BASE_URL`, `LLM_MODEL`, `LLM_FALLBACK_MODELS` in `.env`). Without a key, a labelled deterministic baseline planner runs the same tools. Confirmation is human-only, payload-hash checked and single-use.

10 Human review and sign-off

IN PLAIN WORDS

Machines flag; people decide. For each finding the reviewer picks an outcome and writes the reason. If money is involved, they record the amount and the evidence behind it. Every decision is a new entry; old ones are never overwritten.

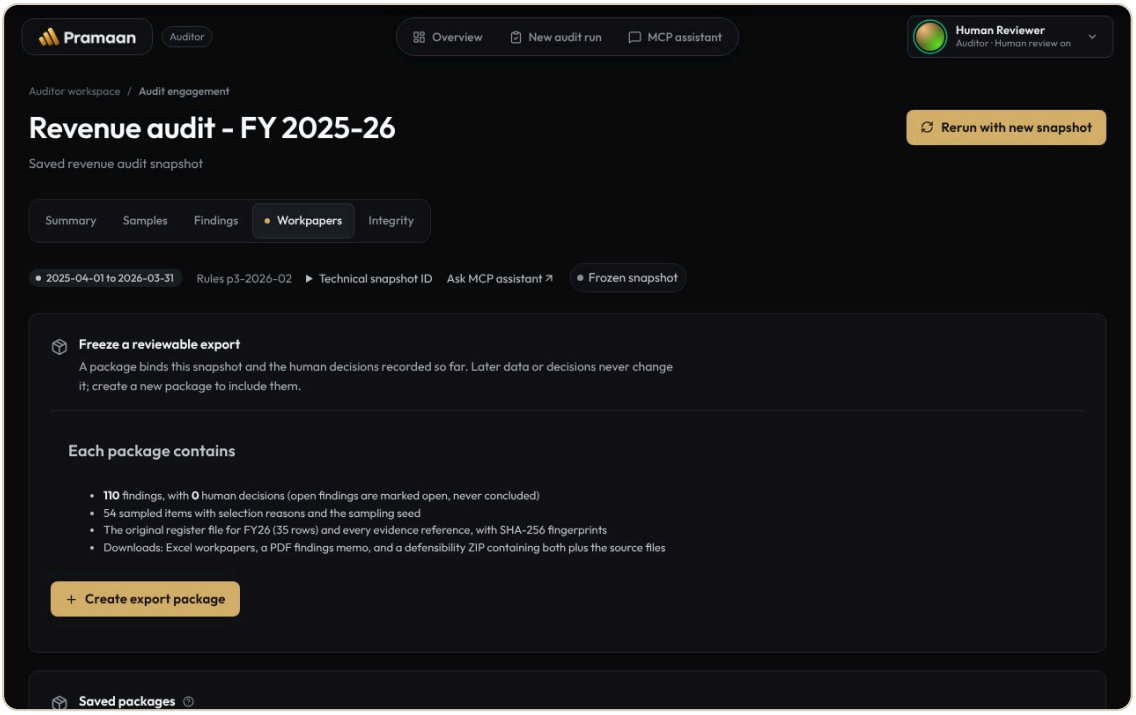
Outcome	Meaning	Required
Exception confirmed	The issue is real	Reason; for quantified impact also a non-negative INR amount, supporting evidence IDs and an impact group
Explained	There is a valid explanation	Reason
Data error	The signal came from bad or incomplete data	Reason

An auditor (or the AI) can **propose** a decision; only the human-review key can **record** it. Several findings can be decided together with bulk review.

UNDER THE HOOD

Decisions are revisioned (each needs the expected revision, which prevents lost updates) and appended, never updated in place. Human-review keys expire within 7 days, can never be permanent or agent-bound, and are refused on the MCP channel. These are human assertions; PRAMAAN does not post journals or verify accounting treatment automatically.

11 Workpapers and export



Workpapers: freeze a reviewable export. A package binds this snapshot and the human decisions so far; later changes need a new package.

Output	Contents
Excel workpapers	A contents sheet linking every sheet: register, sales/cut-off/credit-note workpapers, findings, summary, impact groups, import metadata, receipt, provenance, coverage and tickmarks. Original register cells are kept as parsed.
PDF memo	Frozen findings and reviews, reconciled summary data, plain-language check results, and full hashes in a reproducibility appendix.
Defensibility ZIP	The exact uploaded source bytes, workbook, memo, full report, evidence index, standards candidates (for CA review), integrity verification and receipt.

IN PLAIN WORDS

A package is like a sealed evidence bag: downloading it twice gives byte-identical files, and if anyone edits a file inside, its fingerprint no longer matches the receipt.

12 Integrity checks: how tampering is detected

#1 Register uploaded
`prev: 0000…`
`hash: 7f3a…`

→

#2 Audit run created
`prev: 7f3a…`
`hash: c21e…`

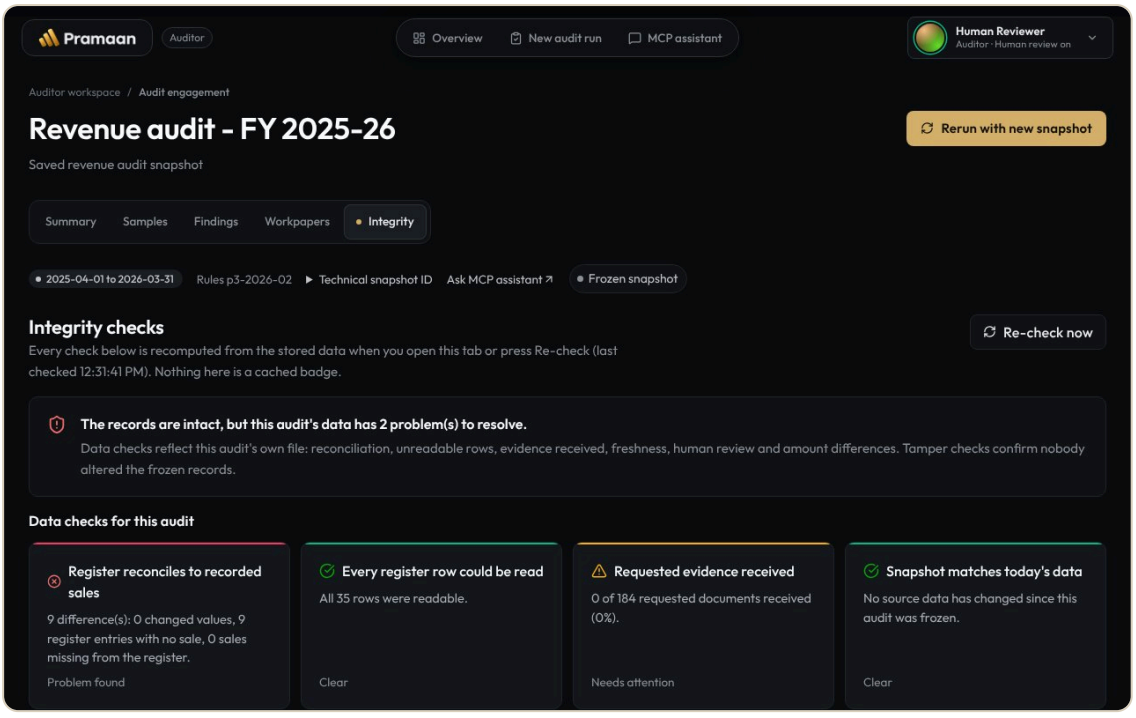
→

#3 Agent step
`prev: c21e…`
`hash: 9b04…`

→

#4 Human decision
`prev: 9b04…`
`hash: e6d8…`

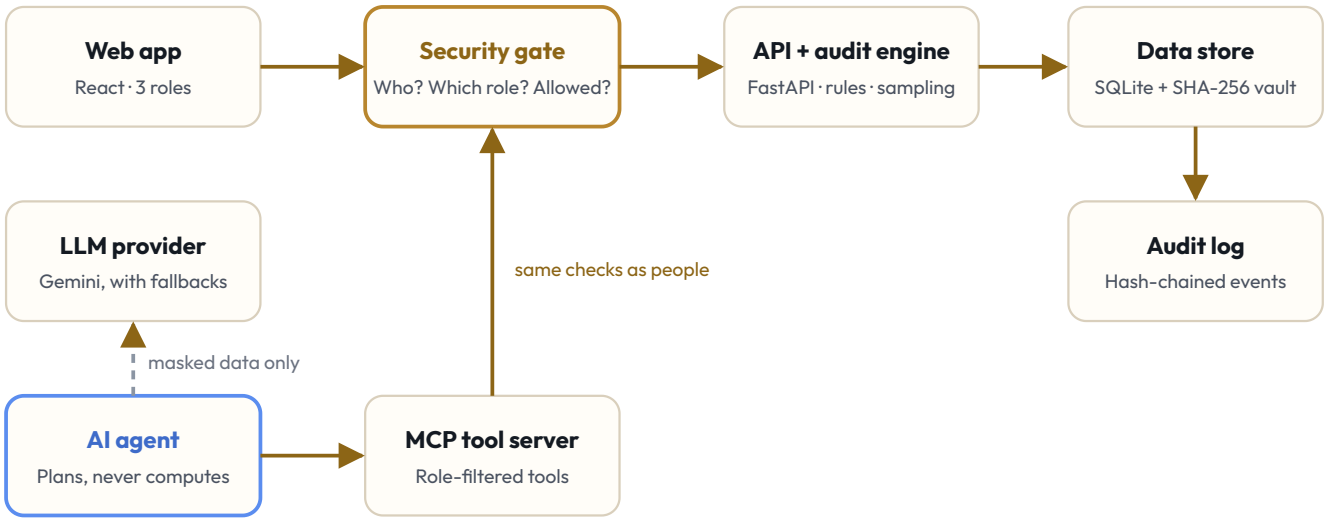
Each event stores the fingerprint of the one before it. Editing #2 changes its hash, so #3's “prev” no longer matches and the chain breaks.



Integrity tab: every check is recomputed when you open it, never a cached badge. Data checks cover the audit's own file; tamper checks confirm nobody altered the frozen records.

Mechanism	Proves	Does not prove
SHA-256 vault	A file is exactly the one received	Who wrote it, or that it is authentic
Immutable snapshot	The run can be reproduced exactly	Anything about changes made after the run
Hash-chained log	History wasn't edited or deleted	Protection against someone replacing the whole database and chain (needs external anchoring)
Demo e-invoice signature	Real Ed25519 verification against a test key	Official NIC/IRP status

13 Architecture



IN PLAIN WORDS

People use the website; the AI uses a tool server. **Both go through the same security gate** before reaching the engine and the data. The AI provider only ever receives masked data.

Component	Where	Responsibility
Web app	`web/`	Client and auditor workspaces. No direct database access.
API + policy	`backend/app.py`, `security.py`	Authenticated routes, role checks, orchestration, UI read models
Audit services	`p2.py`, `p3.py`, `p5.py`	Lifecycles and reconciliation; snapshot tests and sampling; review and packages
Agent	`p6_agent.py`, `party_match.py`, `agent_llm.py`	Investigation loop, payer matching, LLM client with fallbacks
MCP server	`mcp_server.py`	Official MCP SDK (stdio or HTTP); delegates every call to the API; egress DLP
Storage	SQLite + vault	WAL database with append-only triggers; files addressed by SHA-256

Design rule	Meaning
Preserve originals	A revised register is a new upload, never an overwrite
Deterministic core	Checks, snapshots, reconciliation and sampling never depend on an LLM
Separate authorities	Client writes, auditor investigation, proposals and final review are separate operations
Frozen outputs	Packages come from stored snapshot and review data; live changes need a new package

14 Security and privacy

Area	Control
Identity	Keys stored only as hashes; 15-minute default lifetime, 30-day maximum; revocable
Account sign-in (MFA)	Optional named accounts: password (scrypt, 12+ characters) plus a 6-digit authenticator code; codes can't be reused; 5 failures lock the account for 15 minutes; 10 attempts a minute per address; same message for every failure; 8-hour sessions ended on sign-out
Evidence at rest	With a vault key kept outside the data folder, every evidence file is sealed with AES-256-GCM; a missing or wrong key locks the vault instead of serving anything
Backups	One passphrase-encrypted file (databases + evidence); tampering and truncation detected; restore verifies every file and only writes into an empty folder; old backups rotated
Authorization	Every route mapped to a named action and checked per role on every request; unknown routes denied; a SQLite authorizer is a second layer
Human decisions	A separate human-review key; refused on the AI (MCP) channel
AI data egress	GSTIN, PAN, account numbers, phone and email redacted from tool results; account numbers block the whole result. PRAMAAN's own IDs are recognised so they're never mistaken for account numbers
Untrusted documents	Document text reaches agents labelled as untrusted; instructions inside it are not followed
Repository secrets	`.env` and AI-tool folders git-ignored; the Secret scan workflow runs gitleaks over the full history, with extra rules for inline passwords
Dependencies	Pinned; npm audit and pip-audit report 0 known vulnerabilities

Setting up accounts, encryption and backups (Windows)

Run these in PowerShell from the project folder. The launcher applies the right settings and data folder itself. In Command Prompt, leave out the `.\`.

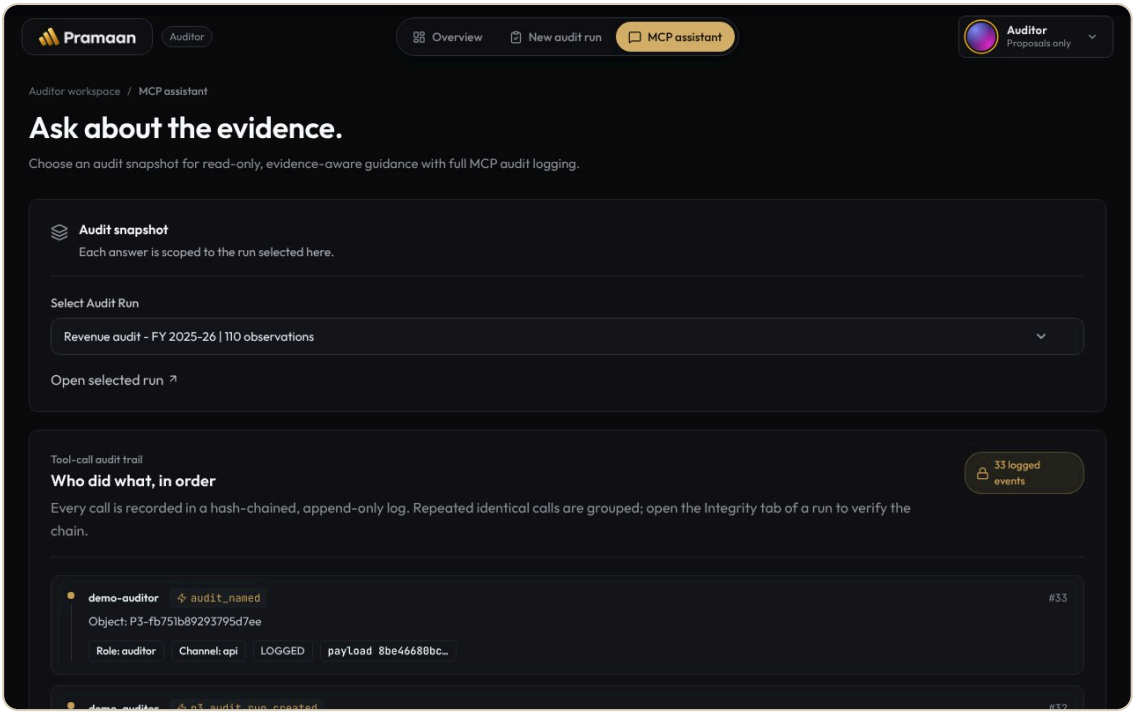
```
.\START_PRAMAAN.bat accounts add username --role auditor
.\START_PRAMAAN.bat accounts list
.\START_PRAMAAN.bat vault new-key C:\pramaan-keys\vault.key
.\START_PRAMAAN.bat backup create --out D:\pramaan-backups --keep 14
```

1. Replace `username` with the login name you want (`--role client_employee` for a client login). You choose the password when asked. PRAMAAN never hands them out.
2. Add the printed key to Google or Microsoft Authenticator ("Enter a setup key"). The phone shows the 6-digit code.
3. Reload the sign-in page. It now opens on **Account**; launcher keys still work under **Credential**.

Known limits (stated plainly)

Designed for one company on one machine with synthetic data. It does **not** provide multi-tenant isolation, OAuth/SSO, distributed rate limiting, database grants, encryption of the SQLite databases (use BitLocker), scheduled off-site backups, an external security review, external anchoring of the hash chain, or official GST/IRP integration. Report vulnerabilities privately (see `SECURITY.md`).

15 AI assistant and MCP tools



MCP assistant: ask questions about one audit run. Answers use read-only tools, and every tool call is recorded in the hash-chained log.

IN PLAIN WORDS

MCP is a standard way for AI assistants to use a program's functions. PRAMAAN offers its functions as MCP tools, so an AI can look things up, but only the tools the key's role allows, and with exactly the same checks a person faces.

Tool group	Examples
Audit investigation	get_findings, select_sales_tod, select_cutoff_tod, get_sale_lifecycle, trace_related_parties, get_journal
Audit execution	create_audit_run, run_tests, reproduce_workpaper, verify_audit_log
Review / exports	get_review_queue, propose_review_decision, create_export_package, export_memo
Client sources	upload_sales_register, upload_related_parties, respond_to_evidence_request
Agent	investigate_finding

UNDER THE HOOD

Official MCP Python SDK over stdio, or local Streamable HTTP at `http://127.0.0.1:8001/mcp` with a fresh bearer key on every request. Tool lists are role-filtered, schemas reject extra properties, and lists are paginated (limit 20, maximum 100). Tools that change client books are denied to agents unless `PRAMAAN_MCP_BOOKS_WRITES=allow`, and every attempt is logged. Hosted third-party MCP clients are not verified.

16 Measured results

IN PLAIN WORDS

We planted known problems in made-up data and measured how many PRAMAAN found, how many false alarms it raised, and whether the AI ever made up numbers. The answer key is used only to grade the results; the software never sees it.

Audit engine (synthetic benchmark)

Measure	Result
Planted irregularities caught	**8 / 8** (simple document-matching baseline: 3 / 8)
Hard-conflict false alarms on innocent sales	**0**
Review-only flags on innocent sales	2 (threshold patterns, not exceptions)

Investigation agent (26 planted cases)

Metric	No-AI planner	Gemini planner
Traced to a source record	26 / 26	26 / 26
Invented amounts	0	0
Correct outcome	25 / 26	25 / 26
Planted conflicts escalated	5 / 6	5 / 6
False escalations	0	0
Same outcome on 5 independent runs	26 / 26	26 / 26 *

* In 2 of 130 investigations the AI reported “not explained” without drafting the evidence request; the outcome class was the same. Model: gemini-3.1-flash-lite-preview with fallbacks; prompt investigate-v2.

The AI matching the fixed baseline is the point: **safety comes from the design** (tools compute, server rules decide, humans confirm), not from trusting the model. The single miss is a look-alike payer name (“Banyan Retail” vs “Verma Retail”) treated as unpaid instead of being flagged as a conflict.

Synthetic data only. Seconds per case measure machine time; no auditor time saving is claimed, because no manual baseline has been timed.

17 Quality and testing

Check	Count / result	Runs
Backend tests (pytest)	253 passing (9 need Tesseract)	Every push (GitHub Actions CI)
UI unit tests (vitest)	39 passing	Every push
Browser journeys (Playwright)	8 passing: keyboard-only review, export, agent investigation, 360 px overflow…	Every push
Accessibility (axe, WCAG 2.1 AA)	0 violations on core screens	UI audit
Automated rehearsal + release verification	Pass	Every push
Agent evaluation (offline baseline)	Pass	Every push
Secret scan (gitleaks, full history)	Clean	Every push
Dependency audit (npm, pip)	0 known vulnerabilities	On demand

IN PLAIN WORDS

Every time code is pushed to GitHub, all of these run automatically on GitHub's computers. A red ❌ appears if anything breaks, so problems are caught straight away instead of later.

Run them yourself

```
$env:PYTHONPATH = "$PWD\launcher"
.\.venv-p4\Scripts\python.exe -m pytest backend/tests -q
cd web
npm run typecheck; npm test; npm run build; npm run test:e2e
```

18 Configuration and command cheat-sheet

Setting	Default	Meaning
PRAMAAN_DATA	backend/data/	Storage folder; use the same value for the API and every command
PYTHONPATH	unset	Absolute path of the `launcher/` folder
LLM_BASE_URL / LLM_MODEL	set in .env	OpenAI-compatible endpoint and model for the agent
LLM_FALLBACK_MODELS	optional	Models tried when the main one is rate-limited or down
PRAMAAN_MAX_UPLOAD_BYTES	10485760	Maximum upload (10 MiB)
PRAMAAN_MCP_PORT	8001	Streamable HTTP MCP port
PRAMAAN_MCP_BOOKS_WRITES	deny	Set to `allow` to let agents write client books

Task	Command (from the project folder, after setting PYTHONPATH)
Reset demo data	`python -m Pramaan.backend.demo reset`
Issue a key	`python -m Pramaan.backend.security issue --subject me --role auditor --human-auditor --ttl 28800`
Agent-only key	`python -m Pramaan.backend.security issue --subject my-agent --role auditor --agent --ttl 900`
Revoke keys	`python -m Pramaan.backend.security revoke-subject --subject my-agent`
Engine evaluation	`python -m Pramaan.backend.evaluate`
Agent evaluation	`python -m Pramaan.backend.evaluate_agent --planner llm --repeats 5 --write-docs`
Seed agent demo	`python -m Pramaan.backend.evaluate_agent --seed-instance .demo/agentx`
MCP over HTTP	`python -m Pramaan.backend.mcp_server --http`
Add an account (MFA)	`.\START_PRAMAAN.bat accounts add username --role auditor`
Unlock / lost phone	`.\START_PRAMAAN.bat accounts unlock NAME` · `accounts reset-mfa NAME`
Encrypted backup	`.\START_PRAMAAN.bat backup create --out D:\backups` · `backup verify FILE` · `backup restore FILE --to DIR`
Vault encryption	`.\START_PRAMAAN.bat vault new-key PATH`, set `PRAMAAN_VAULT_KEY_FILE`, then `vault encrypt`
Secret scan	`gitleaks git --config .gitleaks.toml .`

19 Limits, roadmap and troubleshooting

Honest limits today

- Tested on synthetic data only
- One computer, no hosting; accounts with MFA and encrypted backups exist but have had no external security review
- Tally live fetch exists but is untested against real TallyPrime; no SAP/ERP connectors
- Scanned PDFs need Tesseract installed; tested on synthetic documents only
- Simple payer matching (look-alike names can slip through)
- Bulk review processes one finding at a time behind the scenes
- Reruns don't reuse evidence already uploaded

Roadmap

1. **Now:** working prototype
2. **Next:** pilot with a practising CA on anonymised books
3. **Then:** verify the Tally link on a real TallyPrime, then SAP; test readers on real anonymised statements
4. **Later:** SSO, scheduled backups, hosting, external security review, data-protection compliance

Troubleshooting

Problem	Fix
`Failed to resolve import "@fontsource/…"`	Close the windows and run the launcher again; it reinstalls missing packages
"Your session ended"	The key expired (8 hours). Restart the launcher for new keys
`No module named 'Pramaan'`	From the project folder, run `$env:PYTHONPATH = "$PWD\launcher"` first
Port 8000 or 5173 in use	Close the old PRAMAAN windows, or restart the computer
Want fresh demo data	Delete `.demo\runtime` and start the launcher again
AI says "no LLM key"	Copy `.env.example` to `.env`, add a key, restart; or use "Run without AI"

The AI investigates. The code computes. The human decides. Everything is provable.

PRAMAAN supports an auditor's review; it is not an audit opinion. Open source under AGPL-3.0-only. More detail: README.md, docs/GETTING_STARTED.md, SECURITY.md, CONTRIBUTING.md, CHANGELOG.md.